CS482: Performance Evaluation & Optimization

180-Minute Solo Laboratory Practicum: Empirical Memory Hierarchy, Concurrency Contention, and Scaling Limits

DURATION	FORMAT	SUBMISSION MODE	PRIMARY STACK
180 Minutes (3 Hours)	**Solo • In-Lab • Zero-Paperwork**	**Git Tagged Commits Only**	**Python 3.10+ (Standard Lib + NumPy)**
TOTAL WEIGHT	**ALLOWED RESOURCES**	**STRICT INTEGRITY POLICY**	
100 Points (10% Course Grade)	**Man pages, Python docs, Lecture slides**	**No AI code generation • No peer collaboration • Authenticated hardware profiling**	

ANTI-PLAGIARISM & AI-RESISTANCE PROTOCOL

This laboratory is engineered to be entirely immune to generative AI shortcuts. Every student executes experiments with **dynamically salted problem dimensions** derived from their unique Student ID. Furthermore, grading requires **raw empirical telemetry files** (machine topology, cache hit rates, nanosecond-resolution timings, and hardware performance metrics) gathered directly from your physical CPU. AI cannot predict or simulate your specific CPU's cache line eviction behavior or inter-core coherence latencies. All code, raw data tables, and commit hashes are verified automatically upon repository push.

Overview & Architectural Objective

Throwing CPU cores at unoptimized software often produces sublinear speedups or outright performance regression. In this 3-hour practicum, you will step through the physical hierarchy of modern multi-core computing. You will expose memory-stride pathologies, deliberately trigger cache-line false sharing across physical cores in Python shared memory, compute formal diagnostic scaling metrics (Amdahl, Gustafson, Karp-Flatt), and evaluate your host architecture against the Roofline model.

Laboratory Timeline & Deliverable Summary

Milestone	Suggested Time	Core Focus & Empirical Experiment	Git Commit Tag	Points
Milestone 0	00:00 – 00:15 (15 min)	Hardware topology profiling, parameter salting script	`tag: M0-ENV`	10 pts
Milestone 1	00:15 – 01:00 (45 min)	Spatial locality, stride cliff, loop interchange (i-j-k vs i-k-j)	`tag: M1-LOCALITY`	25 pts
Milestone 2	01:00 – 01:45 (45 min)	Shared memory concurrency, false sharing collapse & Karp-Flatt	`tag: M2-SHARING`	30 pts
Milestone 3	01:45 – 02:35 (50 min)	Strong/Weak scaling studies, Amdahl/ Gustafson fitting, Roofline	`tag: M3-SCALING`	25 pts
Milestone 4	02:35 – 03:00 (25 min)	Automated numerical verification, Git provenance push	`tag: M4-FINAL`	10 pts

Milestone 0: Host Topology Profiling & Salting (15 min) 10 POINTS

Before running benchmarks, you must establish your hardware baseline. You will implement a Python script `profile_env.py` that introspects your physical platform and derives your salted problem parameters.

Task 0.1: Machine Telemetry Extraction

Create `src/profile_env.py`. The script must inspect the host OS and emit a structured JSON file at `results/host_profile.json` recording:

- CPU Model name, microarchitecture, and base frequency (from `/proc/cpuinfo` on Linux, `sysctl` on macOS, or `platform` / `psutil`).
- Physical core count vs. logical execution threads (detecting SMT / Hyper-Threading).
- L1 Data cache size, L2 cache size, L3 shared cache size, and cache-line size (typically 64 bytes).
- Operating system kernel version and Python implementation details.

Task 0.2: Deterministic Student ID Salting

To prevent peer collusion and synthetic AI submissions, each student's problem dimensions are cryptographically bound to their 8-digit Student ID. Implement the following function in `src/salt.py`:

```
import hashlib, os

def get_student_params(student_id: str):
    h = int(hashlib.sha256(student_id.encode('utf-8')).hexdigest()[:8], 16)
    # Salted matrix dimension N between 1024 and 1536 (multiple of 64)
    N = 1024 + ((h % 9) * 64)
    # Salted stride benchmark buffer size: 2^24 elements (~134 MB double-precision)
    buffer_elements = 16_777_216
    # Seed for deterministic random number generation
    seed = h % 100_007
    return {"student_id": student_id, "N": N, "buffer_elements": buffer_elements, "seed": seed}
```

MILESTONE 0 DELIVERABLE CHECKPOINT

Execute `python3 src/profile_env.py --student-id <YOUR_ID>`.
Verify that `results/host_profile.json` and `results/salted_params.json` exist.
Execute:
`git add . && git commit -m "M0: Environment and salting configuration" && git tag M0-ENV`

Milestone 1: Memory Stride & Loop Interchange (45 min) 25 POINTS

Slide 14 and 33 emphasized: *Hardware moves 64-byte cache lines, not individual numbers.* Stepping contiguously across memory (stride-1) exploits prefetchers and consumes all 8 double-precision floats in a cache line. Stepping down columns (stride-N) discards 7 out of 8 fetched doubles, bottlenecking execution on DRAM bus latency.

Task 1.1: Stride Latency Cliff Benchmark

Write `src/stride_bench.py`. Allocate a 1D contiguous array of double-precision floats (64-bit, 8 bytes each) containing 2^{24} elements (~134 MB, intentionally exceeding L3 cache capacity):

```
import time, numpy as np

# Allocate flat array in physical memory
data = np.ones(16_777_216, dtype=np.float64)
```

```
# Measure read throughput for stride s in [1, 2, 4, 8, 16, 32, 64, 128]
# Run 5 warmup iterations, followed by 5 recorded iterations. Log median time_ns.
```

For each stride $s \in \{1, 2, 4, 8, 16, 32, 64, 128\}$, traverse the array accumulating a reduction: `total += data[i]` with step s. Normalize the elapsed wall-clock time per element read:

$$ ext{Latency per element (ns)} = rac{ ext{Elapsed Time (ns)}}{ ext{Total Elements Touched}} $$

Deliverable: Emit `results/stride_results.tsv`. Your data must clearly exhibit the "cache-line cliff" between stride-1 (where each cache miss serves 8 sequential elements) and stride-8+ (where every single memory read incurs a distinct cache line request).

Task 1.2: Loop Interchange on Dense Kernel (i-j-k vs. i-k-j)

Implement `src/dense_kernel.py`. Using your salted dimension N, initialize matrices $A, B \in \mathbb{R}^{N imes N}$ using NumPy's flat buffer or native flat 1D lists.

1. **Algorithm A (Naïve i-j-k):** Loop i from $0 \dots N-1$, loop j from $0 \dots N-1$, loop k from $0 \dots N-1$. Accumulate $C[i, j] += A[i, k] imes B[k, j]$. Notice that $B[k, j]$ strides by N elements on every inner step.
2. **Algorithm B (Interchanged i-k-j):** Loop i from $0 \dots N-1$, loop k from $0 \dots N-1$, loop j from $0 \dots N-1$. Hoist $A[i, k]$ into a register-local variable, and update row $C[i, :]$ by scaling row $B[k, :]$ with stride-1 contiguous access.

Execute both algorithms over a sub-block of size $M = N / 2$ (to fit the 45-minute window if running pure Python, or compile via Cython/Numba/ctypes if specified). Measure median wall-clock runtime across 3 runs. Compute the empirical speedup $S = T_{ ext{naive}} / T_{ ext{interchanged}}$.

> **MILESTONE 1 DELIVERABLE CHECKPOINT**
> Ensure `results/stride_results.tsv` and `results/loop_interchange.json` are populated.
> Execute: `git add . && git commit -m "M1: Memory stride and loop interchange benchmarking" && git tag M1-LOCALITY`

Milestone 2: Concurrency & False Sharing Pathology (45 min) 30 POINTS

Slide 15 detailed the "silent performance killer": independent threads updating variables that reside on the same 64-byte cache line. The hardware's MESI cache-coherence protocol constantly invalidates private L1 caches, ping-ponging the memory line across cores and severely degrading parallel throughput.

> **AI TRAP & PATHOLOGY REPRODUCTION**
> Standard AI code generators always output clean, parallel code that bypasses this problem. In this milestone, **you must deliberately construct the pathological case**, empirically capture the performance collapse, and prove the recovery using the Karp-Flatt metric. Submissions without empirical collapse data will be awarded zero points.

Task 2.1: The Pathological Contention Script

Implement `src/false_sharing.py` using Python's `multiprocessing.shared_memory` or `multiprocessing.Array`:

```
import multiprocessing as mp, ctypes, time

def worker_unpadded(counter_array, worker_id, iterations):
    # Pathological: counter_array is a contiguous array of 64-bit integers.
    # worker 0 writes to index 0; worker 1 writes to index 1.
    # Indices 0 through 7 reside on the EXACT SAME 64-byte physical cache line!
    for _ in range(iterations):
        counter_array[worker_id] += 1
```

- Allocate a shared array of 64-bit signed integers: `shared_arr = mp.Array(ctypes.c_int64, 8)`.
- Spawn P processes ($P \in \{1, 2, 4, 8\}$). Assign worker p to increment `shared_arr[p]` for $K = 5,000,000$ iterations.
- Record runtime $T_{ ext{unpadded}}(P)$ for each core count. Notice how adding cores increases runtime (negative speedup) due to cache-line ping-ponging!

Task 2.2: Remediation via 64-Byte Padding and Privatization

In the same script, implement the two remediation strategies taught in Slide 16:

1. **64-Byte Cache-Line Padding:** Separate each counter by 8 integers ($8 imes 8 ext{ bytes} = 64 ext{ bytes}$). Worker p now writes exclusively to `shared_arr[p * 8]`. Each worker's counter occupies a physically distinct cache line.
2. **Thread-Local Privatization:** Each worker increments a local scalar variable inside its private stack/register frame K times. Only upon loop completion does each process perform a single synchronized write to the shared array.

Measure $T_{ ext{padded}}(P)$ and $T_{ ext{privatized}}(P)$ for $P \in \{1, 2, 4, 8\}$.

Task 2.3: Quantitative Diagnosis via the Karp-Flatt Metric

From your measured runtimes, calculate the empirical Karp-Flatt metric $e(P)$ for both the unpadded and padded implementations across $P \in \{2, 4, 8\}$:

$$e(P) = rac{ rac{1}{S_P} - rac{1}{P}}{1 - rac{1}{P}} = rac{ rac{P \cdot T_P}{T_1} - 1}{P - 1}$$

Theoretical Expectation: In the unpadded case, $e(P)$ will scale upward sharply as P grows, diagnosing severe inter-core hardware contention. In the padded/privatized case, $e(P)$ remains flat and low.

> **MILESTONE 2 DELIVERABLE CHECKPOINT**

> Record all runs in `results/false_sharing_karp_flatt.tsv`.
> Execute: `git add . && git commit -m "M2: False sharing reproduction and Karp-Flatt analysis" && git tag M2-SHARING`

Milestone 3: Scaling Models & Roofline Evaluation (50 min) 25 POINTS

Slide 06 through 09 contrast Amdahl's Law (strong scaling: fixed problem size) with Gustafson's Law (weak scaling: work grows with core capacity). In this milestone, you will evaluate a parallel domain decomposition kernel across multiple worker processes.

Task 3.1: Strong Scaling Study & Amdahl Parameter Extraction

Implement `src/scaling_study.py`. Using `multiprocessing.Pool` or `Process` workers, partition a 2D matrix stencil update of fixed size $N_{ ext{salted}} imes N_{ ext{salted}}$ across chunks:

- Measure execution time T_P for $P \in \{1, 2, 4, \dots, ext{max available physical cores}\}$.
- Calculate Speedup $S_P = T_1 / T_P$ and Parallel Efficiency $E_P = S_P / P imes 100\%$.
- Fit Amdahl's Law to your measurements to extract your program's effective serial fraction s:

$$S_P = rac{1}{s + rac{1 - s}{P}} \implies s = rac{ rac{1}{S_P} - rac{1}{P}}{1 - rac{1}{P}}$$

- Compute your theoretical maximum asymptotic speedup ceiling: $S_{\max} = 1 / s$.

Task 3.2: Weak Scaling Study & Gustafson-Barsis Verification

Now scale your workload proportionally with the processor count. For P processes, set the problem dimension to $N_P = \lceil N imes \sqrt{P} ceil$, ensuring total work scales as $O(P)$:

- Measure parallel wall-clock runtime $T_P^{ ext{weak}}$.
- Compute Scaled Speedup: $S_P^{ ext{scaled}} = rac{P \cdot ext{Work}(1) \cdot T_1}{ ext{Work}(P) \cdot T_P^{ ext{weak}}}$.
- Contrast the strong scaling efficiency drop against the weak scaling throughput.

Task 3.3: Arithmetic Intensity & Roofline Positioning

Calculate the theoretical Arithmetic Intensity (AI) of your stencil kernel:

$$ ext{Arithmetic Intensity (AI)} = rac{ ext{Floating Point Operations (FLOPs)}}{ ext{DRAM Bytes Transferred}}$$

Based on your L1/L2/L3 cache sizes extracted in Milestone 0 and your CPU's peak theoretical memory bandwidth, determine whether your execution regime is **Memory-Bandwidth Bound** or **Compute Bound** (Slide 18–19). Document your conclusion in `results/roofline_summary.json`.

> **MILESTONE 3 DELIVERABLE CHECKPOINT**
> Ensure `results/scaling_metrics.tsv` and `results/roofline_summary.json` are populated.
> Execute: `git add . && git commit -m "M3: Strong and weak scaling evaluation" && git tag M3-SCALING`

Milestone 4: Numerical Verification & Git Submission (25 min) 10 POINTS

Slide 48, Rule 10: *Fast code that returns incorrect answers is not an optimization.* You must verify that your parallel optimizations produce bitwise or floating-point compliant results.

Task 4.1: Automated SHA-256 Output Verification

Run the provided automated verification script `verify_outputs.py`. It inspects all generated result arrays, computes a salted SHA-256 digest of the numerical values rounded to 6 decimal places, and confirms that:

- The interchanged loop matrix multiplication in Milestone 1 matches the baseline within $\epsilon = 10^{-12}$.
- The unpadded, padded, and privatized counters in Milestone 2 all terminated with exactly $P imes K$ total counts.
- The parallel scaling reductions in Milestone 3 match the single-threaded reference checksum.

```
$ python3 verify_outputs.py
[PASS] Milestone 0: Host topology and salted parameters validated.
[PASS] Milestone 1: Matrix numerical digest matches reference: 4f8b2a19...
[PASS] Milestone 2: Reduction sum exact match: 40000000 == 40000000.
[PASS] Milestone 3: Stencil numerical tolerance verified.
All empirical datasets verified successfully. Emitting verification_receipt.sig.
```

Task 4.2: Final Git Commit and Repository Structure

Verify your final repository structure matches the specification below before pushing to the course Git server:

```
lab-submission/
├── src/
│   ├── profile_env.py          # Milestone 0
│   ├── salt.py                 # Milestone 0
│   ├── stride_bench.py         # Milestone 1
```

```
│   ├── dense_kernel.py          # Milestone 1
│   ├── false_sharing.py         # Milestone 2
│   └── scaling_study.py         # Milestone 3
├── results/
│   ├── host_profile.json        # CPU and cache telemetry
│   ├── salted_params.json       # Unique student parameters
│   ├── stride_results.tsv       # Stride cliff data
│   ├── loop_interchange.json    # i-j-k vs i-k-j timings
│   ├── false_sharing_karp_flatt.tsv # Unpadded vs padded scaling + e(P)
│   ├── scaling_metrics.tsv      # Strong vs weak scaling data
│   ├── roofline_summary.json    # AI calculation and regime classification
│   └── verification_receipt.sig # Cryptographic verification signature
└── README.md                    # Brief 1-paragraph summary of bottleneck findings
```

FINAL SUBMISSION COMMAND SEQUENCE

```
git add .
git commit -m "M4: Final verification and completed practicum submission"
git tag M4-FINAL
git push origin main --tags
```

Evaluation Rubric (100 Points Total)

Component	Criteria & Verification Target	Score
Milestone 0: Host Profile	Accurate CPU topology, cache line sizes, correctly salted parameters for student ID.	10 pts
Milestone 1: Locality & Stride	Clear empirical reproduction of stride-1 to stride-8+ latency cliff; valid i-j-k vs i-k-j speedup.	25 pts
Milestone 2: False Sharing	Proven reproduction of cache-line contention collapse in shared memory; successful remediation; valid Karp-Flatt calculation showing rising e in unpadded vs flat in padded.	30 pts
Milestone 3: Scaling & Roofline	Amdahl strong scaling fit, Gustafson weak scaling comparison, rigorous arithmetic intensity calculation.	25 pts
Milestone 4: Integrity & Git	Passing verification suite (SHA-256 match), all 5 timestamped Git tags present within the 180-min window.	10 pts